ASSIGNMENT 1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50)

Nature of Assignment: Individual

Duration: 1 Week

Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time

complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Namma Metro Smart-Card Fare System

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs 20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge during peak hours (8–10 AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version of the gate software was written in a hurry. Some fares are off by a paisa, the peak check accepts 10 AM as peak and 8 PM as off-peak inconsistently, a negative balance once appeared on a card, and the fare values are typed in several places.

The number of journeys processed may grow:

1 station (20,000 journeys a day) → 1 line (500,000 a day) → whole network (3,000,000 a day)

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants, operators and control structures, implement the fare logic and validate it, so that every fare and balance is exact.

Design Challenge:

The metro is adding a fourth zone, a weekend discount and monthly passes.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects 4 Marks

Bloom's Level: K5 – Evaluate

Analyse the fare problem from the object-oriented point of view.

Discuss:

- The objects in the problem (card, passenger, journey, zone, fare table) and the data and behaviour of each
- How the same problem would be written in a procedural style, and what changes in an object-oriented style
- The basic principles of OOP – encapsulation, abstraction, inheritance and polymorphism – and where each could appear in this system
- The rules every correct journey must satisfy (for example: balance never below Rs 50 after a journey)

You are expected to justify each object and rule, not merely list names.

Alternative Concept and Complexity Analysis

If the fare system were written in a purely procedural style (global variables and free functions) instead of SmartCard, Journey and FareTable classes, what would the solution be? Analyse whether the time per journey or the memory per card changes (both are O(1)), and compare how much code must be changed when one fare rule changes.

Evidence Expected

A clear object-oriented analysis with objects, their data and behaviour, rules and stated assumptions.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly 3 Marks

Bloom's Level: K5 – Evaluate

Define the fares, surcharge and limits so that a revision is made in one place only.

Discuss:

- The five kinds of tokens in your program (keywords, identifiers, constants, operators, punctuators) with examples
- #define, const and constexpr constants, and why constexpr is preferred for the fare table
- enum and enum class for zones, and why enum class is safer
- Naming rules and conventions for identifiers

Alternative Concept and Complexity Analysis

If the fares were written with #define macros instead of constexpr constants and an enum class, what would change? Compare type checking and scope, and analyse the memory used (a macro occupies no storage; a constexpr value usually does not either) and whether the running time is affected.

Evidence Expected

A single, well-named set of constants with a justified choice between #define, const, constexpr and enum class.

TASK 3 – Choose Data Types and Check Type Compatibility 4 Marks

Bloom's Level: K5 – Evaluate

Select a C++ data type for every value in the system.

Discuss:

- The type chosen (bool, char, int, long long, unsigned, float, double, auto) for fares, balances, times, zone numbers and journey counts, with sizeof on your system

- Why money should be stored in integer paise and not as double rupees
- Type compatibility rules, and what the compiler does with int, double and bool values in the same expression
- The ranges from <climits> and whether int is enough for a day's collection across the network

Alternative Concept and Complexity Analysis

If money were stored as double rupees instead of long long paise, what would the solution be? Analyse the time per addition (O(1) in both), the space per value (8 bytes in both), and measure how the rounding error grows when n fares are added.

Evidence Expected

A justified data-type table with sizes and ranges, and evidence of one precision or overflow problem with its resolution.

TASK 4 – Control Implicit Conversions 4 Marks

Bloom's Level: K5 – Evaluate

Identify every place in the program where C++ converts a value from one type to another without being told.

Discuss:

- Integer division and integer-to-double conversion
- Comparison of signed and unsigned values, and the compiler warning it produces
- Narrowing conversions, and how brace initialisation (int x{3.7};) prevents them
- static_cast and when an explicit cast is needed

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

int main()
{
   int a = 7, b = 2;
   double c = a / b;
```

```
    double d = static_cast<double>(a) / b;
    std::cout << c << ' ' << d << '\n';

    char ch = 'A' + 2;
    std::cout << ch << ' ' << ch + 1 << '\n';

    bool flag = 5;
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';

    unsigned int u = 1;
    int s = -1;
    std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

My predicted output (write before running):

Explain each line of the actual output, naming the conversion that took place (integer division, char to int promotion, int to bool, signed to unsigned).

Alternative Concept and Complexity Analysis

If every conversion in the program were written explicitly with static_cast (and brace initialisation were used everywhere), instead of relying on implicit conversions, what would the solution be? Analyse whether the running time or memory changes, and which errors are caught at compile time instead of at run time.

Evidence Expected

Identified conversion points with demonstrated surprises, and corrected code using brace initialisation or static_cast.

TASK 5 – Write Correct Expressions with the Right Precedence 4 Marks

Bloom's Level: K5 – Evaluate

Write the expressions for the peak check, the surcharge and the concession.

Discuss:

- Operator precedence and associativity in each expression, including the conditional operator ?: and compound assignment
- The difference between x++ and ++x, and why i = i++ is a mistake
- The result of / and % with negative operands
- Why (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) is correct and hour >= 8 && hour < 10 || hour >= 17 && hour < 20 is equivalent but harder to read

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

int main()
{
    int x = 5, y = 3;
    std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

    int z = x++ + ++y;
    std::cout << z << ' ' << x << ' ' << y << '\n';

    std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
    std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';

    int k = 0;
    for (int i = 0; i < 10; i += 3) k += i;
    std::cout << k << '\n';
}
```

My predicted output (write before running):

For every line, name the operators involved and the order in which precedence and associativity apply them.

Alternative Concept and Complexity Analysis

If the peak-hour check were done with a lookup table bool peak[24] indexed by the hour, instead of a combined && / || expression, what would the solution be? Analyse the time per check (O(1) in both, but fewer comparisons with the table) and the extra space of 24 bytes.

Evidence Expected

Correct, readable expressions with an operator-by-operator explanation.

TASK 6 – Design the Control Structures 4 Marks

Bloom's Level: K5 – Evaluate

Design the decision and looping logic for computing a fare and processing a day's journeys.

Discuss:

- When to use if–else, a switch on the zone, and the conditional operator
- How a for loop, a range-based for loop and a while loop could process the journeys, and which is best
- Where break and continue are needed (for example, skipping an invalid journey)
- What happens when no case of a switch matches

Write pseudocode for the fare computation following the SIMATS Engineering – Standard for Algorithmic Pseudocode before writing C++ code.

Complete the Code

Complete TODO 1 and TODO 2 below using a switch statement and integer paise, and execute the program.

```
#include <iomanip>

#include <iostream>


enum class Zone { Inner, Middle, Outer };
```

```
constexpr int BASE_PAISE[] = {1000, 2000, 3000};   // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;     // Rs 50 must remain on the card

long long fare(Zone z, bool peak)
{
   // TODO 1: Use a switch on z to pick the base fare from BASE_PAISE, then add
   //   PEAK_SURCHARGE_PERCENT % in peak hours using integer arithmetic only

}

bool canTravel(long long balance, long long f)
{
   // TODO 2: Return true only if the balance left after paying the fare is at least
   //   MIN_BALANCE_PAISE

}

int main()
{
   long long balance = 10000;                    // Rs 100.00 in paise
   struct Trip { Zone zone; bool peak; };
   Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
   for (const Trip& t : trips) {
      long long f = fare(t.zone, t.peak);
      if (canTravel(balance, f)) {
         balance -= f;
         std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                   << std::setfill('0') << f % 100 << std::setfill(' ');
      } else {
         std::cout << "Denied (low balance)";
```

```
    }
    std::cout << "  balance Rs " << balance / 100 << '\n';
  }
}
```

Expected output:

Fare Rs 10.00 balance Rs 90

Fare Rs 36.00 balance Rs 54

Denied (low balance) balance Rs 54

Add a journey at exactly 10 AM and at exactly 8 PM and check that your peak rule treats them correctly.

Alternative Concept and Complexity Analysis

If the base fare were looked up from an array indexed by the zone (constexpr long long FARE[] = {1000, 2000, 3000}) instead of a switch, what would the solution be? Analyse the time per lookup (O(1) in both; the switch may become a jump table), the space of the array, and how each design copes with a fourth zone.

Evidence Expected

Pseudocode and C++ control structures for the fare computation, with justified choices.

TASK 7 – From Procedural to Object-Oriented 4 Marks

Bloom's Level: K5 – Evaluate

Rewrite the fare logic in two styles.

Discuss:

- A procedural version with separate variables and free functions
- An object-oriented version with a SmartCard class that keeps its balance private and offers deduct() and canTravel()
- How the class prevents a negative balance
- Which version is easier to change when the minimum balance changes

Alternative Concept and Complexity Analysis

If the balance were kept in a public data member instead of a private member with deduct() and canTravel(), what would the solution be? Analyse whether time or space changes (they do not), and explain which rule of the system can then be broken by any part of the program.

Evidence Expected

Two working versions and a reasoned comparison of the procedural and object-oriented designs.

TASK 8 – Read and Print Data Safely 4 Marks

Bloom's Level: K5 – Evaluate

Design the input and output of the gate program.

Discuss:

- Reading values with cin and checking cin.fail() when the operator types letters instead of numbers
- Clearing the error state with cin.clear() and discarding the bad input
- Formatting money with <iomanip> (setw, setfill, fixed, setprecision)
- Printing a neat journey report

Alternative Concept and Complexity Analysis

If each input line were read with std::getline() and converted with std::stoi() inside try/catch, instead of reading numbers directly with cin >>, what would the solution be? Analyse the time to process a line of length L (O(L) in both) and the buffer space needed, and compare how each recovers from invalid input.

Evidence Expected

Safe input handling with demonstrated recovery from invalid input, and correctly formatted output.

TASK 9 – Implement and Test the Fare Logic 4 Marks

Bloom's Level: K5 – Evaluate

Implement the complete fare logic and test it.

Test:

- Normal journeys in each zone, peak and off-peak
- Boundary cases: exactly 8:00, 10:00, 17:00 and 20:00; a balance that leaves exactly Rs 50
- Invalid cases: an unknown zone, a negative recharge, letters typed for the hour
- A small set of 5 journeys whose total you calculate by hand

Alternative Concept and Complexity Analysis

If each journey were tested against a table of test cases (inputs and expected fares) in a loop, instead of separate hand-written checks, what would the solution be? Analyse the time for t tests (O(t)) and the space of the table, and how easily a new boundary case can be added.

Evidence Expected

Working source code and a test table with expected and actual results for every case.

TASK 10 – Compare Expectations with Execution 4 Marks

Bloom's Level: K5 – Evaluate

Compare what you expected with what actually happened.

Discuss:

- Whether the double and integer-paise totals for 20,000, 500,000 and 3,000,000 journeys agree, and by how much they differ
- Whether int overflows for the network total
- The compiler warnings you received with -Wall -Wextra and what each meant
- Reasons for any difference

Alternative Concept and Complexity Analysis

If the network total were accumulated in an int, a long long or a double, what would happen at 3,000,000 journeys? Analyse when each type fails (overflow versus loss of precision), and compare the time per addition and the space per variable.

Evidence Expected

A reasoned comparison of expected and observed behaviour, supported by measured totals.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the final design for the metro.

Discuss:

- The types, constants and classes you recommend and why
- How a fare revision will be carried out
- The limitations of your design
- Whether the design remains correct for the whole network

Your recommendation must be based on evidence from your own experiments.

Alternative Concept and Complexity Analysis

If the fare table were read from a configuration file when the gate starts, instead of being compiled into the program, what would the solution be? Analyse the extra start-up time to read z fare values and the space they occupy, against the cost of recompiling and redeploying after every revision.

Evidence Expected

A technically justified recommendation supported by your tests and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The metro adds a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones.

Re-evaluate your original design. You may RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis – 2 Marks

Identify which constants, enum values, expressions, switch statements and classes are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether the switch-based fare logic and the SmartCard class can absorb the changes, using worked examples.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe and implement the revised design (for example: a fare table indexed by zone, a separate discount rule, a Pass class).

Evidence Expected

An appropriate design response with working code.

D. Evidence-Based Defence – 2 Marks

Defend your decision using test results and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating tests and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Object-Oriented Analysis	K5 – Evaluate	4
Task 2 – Tokens and Constants	K5 – Evaluate	3
Task 3 – Data Types and Compatibility	K5 – Evaluate	4
Task 4 – Implicit Conversions	K5 – Evaluate	4
Task 5 – Operators and Precedence	K5 – Evaluate	4
Task 6 – Control Structures	K5 – Evaluate	4
Task 7 – Procedural vs Object-Oriented	K5 – Evaluate	4
Task 8 – Input and Output	K5 – Evaluate	4
Task 9 – Implementation & Testing	K5 – Evaluate	4
Task 10 – Expectation vs Execution	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		42
Task 12 – Day-4 Design Challenge	K5 – Evaluate	8
TOTAL		50

Submission Requirements

- GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including UML diagrams, pseudocode, test tables and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

TABLE OF CONTENTS

- Introduction and Problem Overview
- Task 1 – Understand the Problem and Identify the Objects
- Task 2 – Tokens, Keywords, Identifiers and Constants
- Task 3 – Data Types and Type Compatibility
- Task 4 – Implicit Conversions
- Task 5 – Operators and Precedence
- Task 6 – Control Structures and Fare Algorithm
- Task 7 – Procedural and Object-Oriented Implementations
- Task 8 – Safe Input and Formatted Output
- Task 9 – Implementation and Testing
- Task 10 – Expected Behaviour versus Execution
- Task 11 – Final Design Recommendation
- Task 12 – Day-4 Design Challenge
- Overall Conclusion
- Appendix – Consolidated C++ Reference Implementation

Note: Page numbers may change slightly when the document is opened in a different version of Microsoft Word because pagination depends on the installed fonts and printer settings.

INTRODUCTION AND PROBLEM OVERVIEW

The assignment concerns the design of a reliable fare calculation system for a city metro network called the Namma Metro Smart-Card Fare System. The network initially contains three travel zones: Inner, Middle and Outer. The base fares are Rs 10, Rs 20 and Rs 30 respectively. The fare is based on the farthest zone reached during a journey. During peak periods, a surcharge of 20% is added. Peak periods are 8:00 AM to before 10:00 AM and 5:00 PM to before 8:00 PM. Therefore, the start of a peak interval is included and its end is excluded.

Passengers use stored-value smart cards. A journey is permitted only when the balance remaining after the fare is deducted is at least Rs 50. Students and senior citizens are also eligible for concessions, although the original scenario does not specify the exact concession percentage or whether the concession is applied before or after the peak surcharge. A safe implementation must therefore represent the concession as a configurable rule and clearly state the assumption used in any demonstration.

The original software has several defects: floating-point calculations can create one-paisa discrepancies, boundary times may be interpreted inconsistently, a card balance can become negative, and fare values are repeated in several parts of the program. These are not merely coding-style problems. They can cause incorrect collections, unfair passenger treatment and difficulty maintaining the software.

This report evaluates the system through object-oriented design, constants, data types, conversions, operators, control structures, input validation, testing and complexity analysis. Money is stored as integer paise, because one rupee equals 100 paise. This makes fare calculations exact when percentages are calculated using integer arithmetic. The design is then evaluated against a fourth zone, a weekend discount and a monthly pass.

The report distinguishes fixed facts supplied by the assignment from implementation assumptions and measurements. Compiler-dependent sizes and measured totals should be confirmed on the student's own computer. The expected outputs shown here follow the stated code and rules; screenshots, GitHub commits and actual compiler diagnostics must be generated by executing the programs locally.

Assumptions used in this report

- All monetary values are represented in paise; Rs 10 is 1000 paise and Rs 50 is 5000 paise.
- The peak interval is [08:00, 10:00) and [17:00, 20:00), meaning the start is included and the end is excluded.

- The original assignment does not provide the student or senior-citizen concession percentage. The core implementation therefore leaves the concession rate configurable and does not invent a rate.
- A 20% surcharge is calculated as baseFare * 20 / 100 using integer arithmetic. The chosen fare values are divisible sufficiently for the given examples.
- The minimum-balance rule is checked after subtracting the complete fare. A balance of exactly Rs 50 after payment is accepted.
- Where a system-wide total is illustrated, the per-journey fare is explicitly stated so that the arithmetic can be independently checked.

TASK 1 – UNDERSTAND THE PROBLEM AND IDENTIFY THE OBJECTS

1.1 Object-oriented analysis

Object-oriented analysis models the metro system as a group of interacting objects. Each object represents an entity with its own state and responsibilities. This is appropriate because a fare transaction is not just a mathematical expression: it involves a passenger, a card, a journey, a zone and a fare policy. Keeping the data and operations together makes it easier to enforce rules and to extend the system.

Object / Class	Important data (state)	Behaviour (responsibility)
SmartCard	card identifier, balance in paise, status	check available balance, add recharge, approve travel, deduct fare
Passenger	passenger identifier, passenger category, concession eligibility	identify category and applicable concession
Journey	origin, destination/farthest zone, travel time, weekend flag	validate journey details and provide journey facts
Zone	zone enumeration or zone identifier	identify the fare band for the farthest zone travelled
FareTable / FarePolicy	base fares, surcharge rate, time windows, discounts	calculate the correct fare from journey conditions
MetroGate / FareService	references to card, journey and fare policy	coordinate validation, fare calculation and transaction result

SmartCard is a strong class candidate because balance is sensitive data. It should be private so that other parts of the application cannot directly assign a negative value. Instead, recharge() and deduct() methods validate every change. Journey represents one passenger trip and keeps related information together. FareTable centralises the fare values so that a fare revision is made in one location. Passenger is useful because concession eligibility belongs to the passenger rather than to the physical card alone.

Zone is best represented by an enumeration because only a known set of zone values is valid.

1.2 Procedural style compared with OOP

A procedural solution would use variables such as balance, zone, hour and fare, together with free functions such as calculateFare(), isPeakHour() and canTravel(). The main function would read the inputs, call the functions and update the balance. This is straightforward for a small program. However, the variables are not protected as a single unit. Any function could accidentally update balance without checking the minimum-balance rule. If fare rules are repeated in several functions, a change may require editing several locations.

In an object-oriented solution, SmartCard owns its balance and exposes only controlled operations. FarePolicy owns the calculation rules. Journey supplies the journey information. The gate coordinates these objects. This improves cohesion: each class has a clear purpose. It also reduces coupling because the gate does not need to know the internal representation of the card balance.

1.3 Four fundamental OOP principles

Encapsulation means combining data and methods and restricting direct access to internal state. SmartCard stores balance as a private long long value. Its canTravel() method checks whether balance - fare is at least the required minimum. Its deduct() method performs the subtraction only when the check succeeds. This prevents unrelated code from bypassing the rule.

Abstraction means exposing the essential operations while hiding implementation details. A gate can call card.canTravel(fare) without knowing how the card stores money or how the validation is implemented. FarePolicy can expose calculateFare(journey) without forcing the caller to understand every arithmetic step.

Inheritance allows a specialised class to reuse and extend a base class. For example, Passenger could be a base class with StudentPassenger and SeniorPassenger derived classes. Each derived class could supply its concession eligibility or concession calculation. In a small assignment, a PassengerCategory enum may be simpler; inheritance is justified only if the categories have substantially different behaviour.

Polymorphism allows the same interface to invoke different implementations. A virtual concession() method in Passenger could be overridden by StudentPassenger and SeniorPassenger. The fare service could call the same method without checking the concrete passenger type. This is useful when concession policies grow, but it adds

complexity and should be used only when there are multiple genuinely different behaviours.

1.4 Correct-journey invariants and rules

- The zone must be one of the supported values; an unknown numeric zone must be rejected.
- The travel time must be a valid 24-hour time, with hour in the range 0–23 and minute in the range 0–59.
- The peak surcharge applies only when the time is within [08:00,10:00) or [17:00,20:00).
- The base fare is selected from the farthest zone reached, not from the number of stations unless a separate rule is introduced.
- All monetary arithmetic is performed in integer paise and no intermediate operation should silently narrow the value.
- A journey is approved only if balance - fare is greater than or equal to 5000 paise.
- A denied journey must not change the card balance.
- A successful deduction must never produce a negative balance.
- Fare values and policy percentages must be defined once and reused.

1.5 Alternative concept and complexity

A procedural design can store the fare values in a global constant array and keep the balance in a variable. calculateFare(zone, peak) returns the fare and canTravel(balance, fare) checks the remaining amount. For each journey, a fixed number of comparisons and arithmetic operations are performed. Therefore, time per journey is O(1). Each card balance occupies one fixed-size numeric value, so memory per card is O(1) in both procedural and object-oriented designs.

The main difference is maintainability rather than asymptotic performance. If the minimum balance changes, a procedural program can be updated in one constant if all functions use that constant; if the value is repeated, several edits are required. In OOP, the rule can be centralised inside SmartCard or a policy object. A class does not automatically make code correct, but it creates a boundary that helps enforce the invariant. The best choice is to use OOP for card state and fare policy, while keeping small pure helper functions for operations such as isPeakHour().

1.6 Evaluation

The object-oriented design is recommended because the system contains identifiable entities and important invariants. It provides a natural place for card security, fare rules

and journey details. A purely procedural implementation remains acceptable for a tiny demonstration, but it becomes harder to maintain as new passenger categories, zones and discounts are introduced.

TASK 2 – TOKENS, KEYWORDS, IDENTIFIERS AND CONSTANTS

2.1 Five kinds of C++ tokens

A token is the smallest meaningful unit recognised by the C++ compiler. The five groups required in this task are keywords, identifiers, constants or literals, operators and punctuators. Correct token use improves readability and prevents naming conflicts.

Token kind	Meaning	Examples from the fare program
Keywords	Reserved words with predefined language meaning	int, bool, class, enum, return, if, switch, const, constexpr
Identifiers	Programmer-chosen names	SmartCard, balancePaise, calculateFare, PEAK_RATE
Constants / literals	Fixed values written directly or declared as constants	5000, 20, 1000, true, 'A'
Operators	Symbols that perform operations	+, -, *, /, %, ==, >=, &&, ||, +=, ?:, <<
Punctuators	Symbols that structure declarations and statements	{ }, ( ), [ ], ;, ,, :, ::

An identifier must begin with a letter or underscore, followed by letters, digits or underscores. It cannot be a keyword and it is case-sensitive. For example, balancePaise and BalancePaise are different identifiers. Names should communicate purpose: MIN_BALANCE_PAISE is clearer than x or m. A consistent convention such as PascalCase for class names and lowerCamelCase for functions and variables makes the code easier to inspect.

2.2 #define, const and constexpr

#define is a preprocessor directive. It performs textual substitution before compilation. For example, #define MIN_BALANCE 5000 replaces matching tokens with 5000. It does not provide normal C++ type checking or namespace scope. A macro can also cause unexpected substitutions if its name is too common.

const declares an object whose value cannot be modified after initialisation. It has a type and follows C++ scope rules. constexpr declares a value or function that can be evaluated at compile time when its inputs permit it. A fare table with fixed values is a good constexpr candidate because its values are known at compile time and should not change while the program runs.

```
constexpr long long RUPEES_TO_PAISE = 100;
```

```
constexpr long long MIN_BALANCE_PAISE = 50 * RUPEES_TO_PAISE;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
```

The array above is a single source of truth for the three base fares. If the Inner fare changes, only the first array element needs to be updated. A production system may use std::array<long long, 3> for stronger type and size information.

2.3 enum and enum class for zones

A traditional enum introduces enumerator names into the surrounding scope and can implicitly convert to an integer. This can create name collisions and permit accidental arithmetic. enum class is scoped, strongly typed and does not implicitly convert to int. It therefore reduces accidental mixing of a zone with an unrelated integer.

```
enum class Zone { Inner, Middle, Outer };

Zone selected = Zone::Middle;
// int n = selected;              // Not allowed implicitly
int n = static_cast<int>(selected); // Explicit conversion if required
```

The design should avoid accepting arbitrary integer values and treating them as valid zones. Input should first be checked against the supported set and then converted to Zone. When a fourth zone is introduced, the enum and every associated fare lookup must be updated together.

2.4 Naming conventions and single-source configuration

- Use descriptive nouns for data: balancePaise, journeyCount and passengerCategory.
- Use verbs for functions: calculateFare(), addRecharge(), canTravel() and printReceipt().
- Use PascalCase for class and enum type names: SmartCard, FarePolicy and Zone.
- Use UPPER_SNAKE_CASE for project-wide constants if that convention is applied consistently.
- Avoid ambiguous one-letter identifiers except for short loop counters.
- Keep constants close to the policy they describe, or in one dedicated configuration header.

2.5 Alternative concept and complexity

If macros replaced constexpr constants and enum class, the source might become shorter, but type safety would be weaker. A macro has no C++ type, no ordinary scope and no debugger-visible object in the same sense as a variable. constexpr values are typed and checked by the compiler. In many cases, a constexpr scalar does not require a separate run-time storage location because the compiler substitutes or folds its value.

A constexpr array, if actually odr-used, may occupy storage. The macro itself occupies no run-time storage.

Neither approach changes the asymptotic running time of a fare calculation. The compiler generally resolves both constant forms before or during compilation. The important difference is compile-time safety and maintainability. constexpr and enum class are preferred because they make invalid combinations harder to express.

2.6 Evaluation

The recommended implementation uses constexpr for fixed fare values and percentages, enum class for zones, and descriptive identifiers. Macros should be reserved for conditional compilation or other preprocessor-specific tasks, not ordinary fare constants.

TASK 3 – CHOOSE DATA TYPES AND CHECK TYPE COMPATIBILITY

3.1 Data-type selection

A data type determines the set of values a variable can represent, its storage requirements and the operations that are valid. The metro system processes money, time, zone identifiers, passenger counts and transaction totals. Each should use a type that matches its range and meaning.

Value	Recommended type	Reason
Fare and card balance	long long (paise)	Exact integer arithmetic; supports large accumulated values
Peak flag / eligibility	bool	Only true or false are valid
Hour and minute	int or unsigned int after validation	Small whole-number values with clear ranges
Zone	enum class Zone	Restricts the conceptual value to named zones
Journey count	long long or std::uint64_t	Large daily/network totals
Passenger initial	char, if required	A single character, not a full name
Average or statistical percentage	double	Suitable for approximate measurements, not currency storage
Compile-time fare rate	constexpr int	Known fixed policy value
Inferred local variable	auto	Useful when the type is obvious from the initialiser

The exact sizeof values depend on the compiler, target architecture and ABI. On many modern 64-bit desktop systems, sizeof(bool) is 1 byte, sizeof(char) is 1 byte, sizeof(int) is 4 bytes, sizeof(long long) is 8 bytes, sizeof(float) is 4 bytes and sizeof(double) is 8

bytes. These are common results, not universal guarantees for every implementation. The student's program should print sizeof on the actual compiler.

```
#include <iostream>
int main() {
   std::cout << "bool: " << sizeof(bool) << '\\n';
   std::cout << "char: " << sizeof(char) << '\\n';
   std::cout << "int: " << sizeof(int) << '\\n';
   std::cout << "long long: " << sizeof(long long) << '\\n';
   std::cout << "float: " << sizeof(float) << '\\n';
   std::cout << "double: " << sizeof(double) << '\\n';
}
```

3.2 Why money is stored in integer paise

Binary floating-point types such as float and double represent many decimal fractions approximately. A value such as 0.1 cannot be represented exactly as a finite binary fraction. Repeated calculations may therefore produce a value slightly above or below the mathematical result. When the amount is rounded for display, the stored value may differ by one paisa in some cases.

Integer paise avoids this problem for ordinary rupee-and-paise amounts. Rs 36.25 is stored as 3625. Addition and subtraction are exact as long as the chosen integer type does not overflow. Percentage calculations require a defined rounding policy. For a 20% surcharge, base * 20 / 100 gives an exact integer result for the supplied base fares. For arbitrary percentages, the program must specify whether fractional paise are rounded down, rounded to nearest, or handled through a rational calculation.

3.3 Type compatibility and usual conversions

When an int and a double appear in the same arithmetic expression, the int is normally converted to double before the operation. For example, 5 + 2.5 produces a double value 7.5. When a bool is used in arithmetic, false is converted to 0 and true to 1. When an integer is assigned to bool, zero becomes false and any non-zero value becomes true. These conversions are legal but should be understood.

The usual arithmetic conversions also apply to signed and unsigned integers. Comparing a negative signed int with an unsigned int can convert the signed value to unsigned. The resulting value may be a very large positive number, so a comparison such as -1 < 1u can evaluate to false. This is a common source of surprising behaviour.

3.4 Ranges and network totals

The <climits> header provides limits such as INT_MAX and LLONG_MAX. The values should be printed on the target compiler rather than assumed. A common 32-bit int has a maximum value of 2,147,483,647. A long long commonly has a maximum

value of approximately 9.22 × 10^18. The maximum daily collection depends on the fare and the number of journeys.

Journeys per day	At Rs 10 each	At Rs 20 each	At Rs 30 each
20,000	Rs 200,000	Rs 400,000	Rs 600,000
500,000	Rs 5,000,000	Rs 10,000,000	Rs 15,000,000
3,000,000	Rs 30,000,000	Rs 60,000,000	Rs 90,000,000

The largest illustrated total is Rs 90,000,000, equivalent to 9,000,000,000 paise. This exceeds the common signed 32-bit int maximum when stored in paise, but it is comfortably within long long. Therefore, long long is appropriate for aggregate paise totals. A single fare and a card balance are also stored as long long for consistency and future growth.

3.5 Alternative concept: double rupees

If money were stored as double rupees, each addition would still take O(1) time and each double would commonly use 8 bytes. A long long also commonly uses 8 bytes. The storage size and asymptotic complexity are therefore similar, but the numerical guarantees are different. Integer paise represents every whole paisa exactly within range, whereas double represents many decimal amounts approximately.

A rounding experiment can be performed by repeatedly adding 0.1 to a double and comparing the result with an integer counter of 10 paise units. The exact observed difference depends on the sequence and compiler, but a difference may appear because binary floating-point cannot exactly represent 0.1. The assignment's own evidence should include the printed result from the student's compiler. Do not claim a measured error without running this experiment.

3.6 Evaluation

Use long long for monetary values and totals, int for validated time fields, bool for logical states and enum class for zones. Use double only for approximate statistical quantities. This combination provides exact currency handling and sufficient range for the stated network scale.

TASK 4 – CONTROL IMPLICIT CONVERSIONS

4.1 Conversion points in the fare system

An implicit conversion occurs when C++ changes a value from one type to another without an explicit cast. Conversions are sometimes convenient, but they can hide loss

of information or change the meaning of a comparison. The fare program should make important conversions explicit and use types consistently.

Integer division discards the fractional part toward zero. For example, 7 / 2 produces 3 because both operands are integers. Assigning that result to a double does not restore the lost fraction: double value = 7 / 2; stores 3.0. To obtain 3.5, at least one operand must be converted before division, such as static_cast<double>(7) / 2.

A char participates in arithmetic through integral promotion. The expression 'A' + 2 is evaluated as an integer code point value, and assigning the result back to char stores the corresponding character if representable. Printing ch prints the character; printing ch + 1 prints an integer because the arithmetic expression has type int.

An integer assigned to bool is converted to false if it is zero and true otherwise. The stream normally prints bool as 0 or 1. After std::boolalpha is enabled, it prints false or true.

When a signed and unsigned integer are compared, the usual arithmetic conversions may convert the signed value to unsigned. A negative value then wraps to a large unsigned value. This is why signed and unsigned values should not be mixed casually. Compiler warnings such as -Wsign-compare help identify this risk.

4.2 Predict the output before execution

```
#include <iostream>
int main() {
   int a = 7, b = 2;
   double c = a / b;
   double d = static_cast<double>(a) / b;
   std::cout << c << ' ' << d << '\\n';

   char ch = 'A' + 2;
   std::cout << ch << ' ' << ch + 1 << '\\n';

   bool flag = 5;
   std::cout << flag << ' ' << std::boolalpha << flag << '\\n';

   unsigned int u = 1;
   int s = -1;
   std::cout << (s < u ? "s < u" : "s >= u") << '\\n';
}
```

4.3 Exact expected output

```
3 3.5
C 68
1 true
s >= u
```

Explanation of line 1: a / b is integer division because both operands are int. Seven divided by two produces 3, which is then converted to double as 3.0. The second expression converts a to double before division, so it produces 3.5. The default stream formatting prints these as 3 and 3.5.

Explanation of line 2: 'A' + 2 is integer arithmetic after char-to-int promotion. The resulting character code corresponds to C, so ch prints C. In ch + 1, the result is an int and the numeric value of D is 68 in the common ASCII-compatible execution character set.

Explanation of line 3: 5 converts to bool true. Without boolalpha the stream prints 1. With boolalpha it prints true. The manipulator changes subsequent bool formatting for that stream.

Explanation of line 4: s is -1 and u is unsigned 1. The comparison converts s to unsigned. The converted value is a very large unsigned value, so it is not less than 1. The conditional operator therefore selects the text s >= u. This demonstrates why negative signed values should be checked before conversion to unsigned.

4.4 Safer code and explicit conversions

```
int whole = 7 / 2;  // intentional integer division
double exactRatio = static_cast<double>(7) / 2.0;

int hour{};
if (!(std::cin >> hour)) {
   // handle invalid input
}

int narrowed{3};    // safe brace initialisation
// int bad{3.7};    // compile-time error: narrowing is rejected

long long amountPaise = 3625LL;
double amountRupees = static_cast<double>(amountPaise) / 100.0;
```

Brace initialisation prevents many narrowing conversions. For example, int value{3.7}; is ill-formed because converting 3.7 to int would lose information. static_cast documents an intentional conversion, but it does not make an unsafe conversion automatically safe. The programmer must still validate ranges.

4.5 Alternative concept and complexity

If every conversion were explicit, the program would be more verbose but the intent would be easier to review. Brace initialisation would catch narrowing at compile time. Explicit casts would make conversions visible in code review. The running time and memory use would normally remain unchanged because casts between built-in

arithmetic types are simple operations. Some errors are caught at compile time, while invalid user input and out-of-range values still require run-time validation.

4.6 Evaluation

Use consistent types for money, avoid signed/unsigned mixing, validate before casting, use brace initialisation for declarations and apply static_cast where a conversion is intentional. The output prediction should be written before the program is executed, as required by the assignment.

TASK 5 – WRITE CORRECT EXPRESSIONS WITH THE RIGHT PRECEDENCE

5.1 Peak-hour expression

The correct peak-hour condition is (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20). It represents two half-open intervals. At exactly 8, the first interval is true. At exactly 10, it is false. At exactly 17, the second interval is true. At exactly 20, it is false. The use of less-than at the ending boundary prevents 10 AM and 8 PM from being treated as peak.

```
bool isPeakHour(int hour) {
   return (hour >= 8 && hour < 10) | |
         (hour >= 17 && hour < 20);
}
```

The expression hour >= 8 && hour < 10 || hour >= 17 && hour < 20 is equivalent because logical AND has higher precedence than logical OR. However, parentheses improve readability and reduce the chance of a later edit changing the intended grouping.

5.2 Surcharge and concession expressions

```
long long fare = baseFarePaise;
if (peak) {
   fare += baseFarePaise * PEAK_SURCHARGE_PERCENT / 100;
}

// A configurable concession example; rate must come from an approved policy.
long long concessionPaise =
   fare * concessionPercent / 100;
fare -= concessionPaise;
```

The surcharge expression uses multiplication and division before compound assignment. The result of baseFarePaise * 20 / 100 is added to fare. A concession should be represented as a separate policy step. The original assignment states that students and senior citizens receive a concession but does not specify its percentage or

ordering relative to the peak surcharge. The implementation must not silently invent those details. The example therefore uses a configurable concessionPercent and requires the actual approved rate to be supplied.

5.3 Precedence and associativity

Multiplication, division and remainder have higher precedence than addition and subtraction. Relational operators such as < and >= are evaluated after arithmetic. Equality operators follow relational operators. Logical AND has higher precedence than logical OR. The conditional operator ?: has lower precedence than logical OR and evaluates one of its two result expressions depending on the condition. Compound assignment such as += updates the left operand using the result of the right-hand expression.

Parentheses should be used whenever they make business rules easier to understand. They are especially useful for time intervals, percentage calculations and mixed logical expressions. Relying only on the language's precedence table may produce code that is technically correct but difficult to maintain.

5.4 Prefix and postfix increment

The postfix expression x++ yields the old value of x and then increments x. The prefix expression ++x increments x first and yields the new value. In a larger expression, this difference affects the value used. The expression i = i++ should be avoided because it attempts to modify and assign the same variable in a confusing way; historically it has also been associated with undefined behaviour in older language versions. A clear separate statement is safer: ++i;

5.5 Division and remainder with negative operands

For integer division, C++ truncates the quotient toward zero. Thus -10 / 3 is -3. The remainder is defined so that a = (a / b) * b + (a % b), when the quotient is representable. Therefore -10 % 3 is -1. The remainder takes the sign of the dividend. This matters if a program uses modulo to split negative values into time or money components; invalid negative values should normally be rejected before such calculations.

5.6 Predict the output before execution

```
#include <iostream>
int main() {
   int x = 5, y = 3;
   std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\\n';
```

```
    int z = x++ + ++y;
    std::cout << z << ' ' << x << ' ' << y << '\\n';

    std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\\n';
    std::cout << 10 / 3 << ' ' << 10 % 3 << ' '
              << -10 / 3 << ' ' << -10 % 3 << '\\n';

    int k = 0;
    for (int i = 0; i < 10; i += 3) k += i;
    std::cout << k << '\\n';
}
```

5.7 Exact expected output and explanation

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Line 1: multiplication has higher precedence than addition, so 3 * 2 is evaluated first and 5 + 6 gives 11. Parentheses force 5 + 3 to be evaluated before multiplication, giving 16.

Line 2: x++ contributes the old value 5 and then x becomes 6. ++y increments y from 3 to 4 and contributes 4. The sum z is 9. The printed values are therefore 9, 6 and 4.

Line 3: x is 6 and y is 4. The conditional expression selects x because x > y. The remainder and multiplication are evaluated left to right at the same precedence: (6 % 4) * 2 equals 4.

Line 4: integer division gives 3 and remainder 1 for 10 divided by 3. For -10 divided by 3, the quotient truncates toward zero to -3 and the remainder is -1.

Line 5: the loop visits i values 0, 3, 6 and 9. Their sum is 18. The update i += 3 is a compound assignment performed after each loop body.

5.8 Alternative concept: lookup table

A bool peak[24] lookup table can be prepared once, setting true for hours 8, 9, 17, 18 and 19 and false for all other hours. Each query then performs one indexed lookup, which is O(1), just like the combined logical expression. The table uses 24 bool elements; on common systems this is 24 bytes, although sizeof(bool) should be checked on the target implementation. The expression version uses a small fixed number of comparisons and requires no table. A table is useful when the schedule is more complex or configurable, but the direct expression is simpler for two fixed intervals.

5.9 Evaluation

The explicit interval expression is recommended for the original two peak periods. Parentheses make the boundaries clear. The test suite must include exactly 8, 10, 17 and 20 to prove the interval behaviour.

TASK 6 – DESIGN THE CONTROL STRUCTURES

6.1 Choosing decision structures

if–else is appropriate for conditions involving ranges or multiple logical tests. Peak-hour detection, balance validation and input validation naturally use if statements. A switch is suitable for selecting a base fare from a finite set of zones. The conditional operator is suitable for a short expression with two alternatives, but it should not replace a longer decision tree because nested conditional expressions are difficult to read.

A for loop is suitable when the number of journeys is known or stored in a collection. A range-based for loop is often best when processing every element in an array or vector because it avoids manual index management. A while loop is appropriate when processing continues until a sentinel or a condition becomes false, such as reading transactions until the operator enters a stop command.

break exits the nearest loop or switch. In a switch, break prevents execution from falling through into the next case. continue skips the remaining statements in the current loop iteration and proceeds to the next iteration. Invalid journeys should normally be rejected with a clear message and then skipped without changing the card balance.

6.2 Pseudocode for fare computation

```
ALGORITHM CalculateMetroFare
INPUT: zone, hour, minute, cardBalancePaise
OUTPUT: farePaise and transaction status

1. Read zone, hour and minute.
2. If zone is not Inner, Middle or Outer, reject the journey.
3. If hour is outside 0..23 or minute is outside 0..59, reject the journey.
4. Use a selection structure:
     Inner  -> baseFare = 1000
     Middle -> baseFare = 2000
     Outer  -> baseFare = 3000
5. Set peak = false.
6. If (hour >= 8 AND hour < 10) OR
     (hour >= 17 AND hour < 20), set peak = true.
7. Set fare = baseFare.
8. If peak is true, add baseFare * 20 / 100 to fare.
```

9. If cardBalancePaise - fare >= 5000:
   subtract fare from balance;
   print fare and updated balance;
   mark journey as approved.
  Otherwise:
   print low-balance denial;
   leave balance unchanged.
10. Stop.

The pseudocode is deliberately independent of C++ syntax. It describes the required sequence: validate, select the base fare, evaluate peak status, calculate the fare, check the remaining balance and update the card only after approval.

6.3 Complete the code – switch and integer paise

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };

constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak) {
   long long base = 0;
   switch (z) {
      case Zone::Inner:
         base = BASE_PAISE[0];
         break;
      case Zone::Middle:
         base = BASE_PAISE[1];
         break;
      case Zone::Outer:
         base = BASE_PAISE[2];
         break;
      default:
         return -1; // invalid zone sentinel for this demonstration
   }

   if (peak) {
      base += base * PEAK_SURCHARGE_PERCENT / 100;
   }
   return base;
}

bool canTravel(long long balance, long long f) {
   return f >= 0 && balance >= f &&
         balance - f >= MIN_BALANCE_PAISE;
}

int main() {
   long long balance = 10000;
   struct Trip { Zone zone; bool peak; };
   Trip trips[] = {
```

```
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}
    };

    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.'
                      << std::setw(2) << std::setfill('0')
                      << f % 100 << std::setfill(' ');
        } else {
            std::cout << "Denied (low balance)";
        }
        std::cout << " balance Rs " << balance / 100 << '\\n';
    }
}
```

6.4 Expected output

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

The initial balance is Rs 100. The Inner off-peak fare is Rs 10, leaving Rs 90. The Outer peak fare is Rs 30 plus 20%, which is Rs 36, leaving Rs 54. The Middle off-peak fare is Rs 20. Paying it would leave Rs 34, which is below the required Rs 50. The third journey is therefore denied and the balance remains Rs 54.

6.5 Boundary time validation

The hour-only helper correctly classifies 8 and 9 as peak, and 17, 18 and 19 as peak. It classifies 10, 20 and all other hours as off-peak. If the system later needs minute-level boundary precision, represent time as minutes since midnight. The intervals then become [480,600) and [1020,1200). This makes 10:00 and 20:00 explicitly excluded while 08:00 and 17:00 are included.

6.6 Alternative concept: array lookup

An array indexed by zone can replace the switch. The lookup is O(1) and the array occupies three long long elements in the original design. A switch also has O(1) time for a fixed number of cases in the usual complexity model; the compiler may implement it as a jump table or a sequence of comparisons. The array is concise when enum values map directly to indices, but it requires careful validation to prevent out-of-range indexing. A fourth zone requires adding an enum value, a fare entry and updating the array size. A switch requires adding another case. A std::array with a named zone-to-index conversion is safer than raw indexing.

6.7 Evaluation

Use switch for the initial small zone set because it makes each named case visible. Use range-based for to process a known collection of trips. Keep balance deduction inside a guarded branch so that a denied journey never changes the balance.

TASK 7 – FROM PROCEDURAL TO OBJECT-ORIENTED

7.1 Procedural version

In a procedural program, balance is a variable and fare calculation and travel validation are free functions. The program passes the current balance and fare as arguments. This makes the flow easy to follow, but the variable can be changed by any part of the program. The rule that the balance must remain at least Rs 50 after a journey depends on every caller remembering to use the validation function.

```
#include <iostream>

constexpr long long MIN_BALANCE = 5000;

long long calculateSimpleFare(int zone, bool peak) {
   long long base = 0;
   if (zone == 0) base = 1000;
   else if (zone == 1) base = 2000;
   else if (zone == 2) base = 3000;
   else return -1;

   if (peak) base += base * 20 / 100;
   return base;
}

bool canTravelProcedural(long long balance, long long fare) {
   return fare >= 0 && balance - fare >= MIN_BALANCE;
}

int main() {
   long long balance = 10000;
   long long f = calculateSimpleFare(2, true);
   if (canTravelProcedural(balance, f)) {
      balance -= f;
      std::cout << "Approved. Balance paise: " << balance << '\\n';
   } else {
      std::cout << "Denied. Balance paise: " << balance << '\\n';
   }
}
```

7.2 Object-oriented SmartCard class

The object-oriented version stores the balance in a private member. The constructor establishes the initial state. canTravel() checks the rule and deduct() performs the

update only when the rule is satisfied. recharge() rejects negative recharge amounts. This design protects the balance invariant at the point where the state is changed.

```
#include <iostream>

class SmartCard {
private:
   long long balancePaise;
   static constexpr long long MIN_BALANCE_PAISE = 5000;

public:
   explicit SmartCard(long long initialBalance)
      : balancePaise(initialBalance >= 0 ? initialBalance : 0) {}

   long long getBalance() const {
      return balancePaise;
   }

   bool canTravel(long long farePaise) const {
      return farePaise >= 0 &&
            balancePaise >= farePaise &&
            balancePaise - farePaise >= MIN_BALANCE_PAISE;
   }

   bool deduct(long long farePaise) {
      if (!canTravel(farePaise)) return false;
      balancePaise -= farePaise;
      return true;
   }

   bool recharge(long long amountPaise) {
      if (amountPaise <= 0) return false;
      balancePaise += amountPaise;
      return true;
   }
};

int main() {
   SmartCard card(10000);
   if (card.deduct(3600)) {
      std::cout << "Approved. Balance paise: "
            << card.getBalance() << '\\n';
   } else {
      std::cout << "Denied. Balance paise: "
            << card.getBalance() << '\\n';
   }
}
```

The constructor uses an initial-value check to avoid creating a negative balance. In a production application, invalid construction could instead throw an exception or return an error from a factory function. The example keeps the class simple for an introductory assignment.

7.3 How encapsulation prevents negative balances

The balance member is private, so code outside SmartCard cannot write card.balancePaise = -100. The only public method that subtracts money is deduct(). That method first calls canTravel(). It checks fare validity, confirms that the balance can cover the fare and confirms that the remaining amount is at least 5000 paise. If any check fails, deduct() returns false and leaves the balance unchanged.

A class cannot guarantee safety if it exposes another method that bypasses validation. Therefore, every operation that modifies balance must preserve the same invariant. Recharge must reject negative amounts, and any refund method must validate its input. This is an example of designing an object around its invariants.

7.4 Which design handles a minimum-balance revision better?

If the minimum balance changes from Rs 50 to Rs 60, the procedural design is easy to update only when every function refers to one shared constant. If the value has been copied into several functions, each copy must be located and changed. In the class design, the minimum is a class constant and all card checks use it. The change is therefore localised.

7.5 Alternative concept: public balance

If balance were public, any function could directly assign a value or subtract a fare. Time complexity and memory usage would remain the same: reading or writing a numeric member is O(1), and the balance still occupies one numeric value. However, the key system invariant could be broken. A caller could write a negative balance or deduct a fare without preserving the Rs 50 minimum. Private state with validated methods is therefore preferred.

7.6 Evaluation

The procedural version is useful for understanding the algorithm. The SmartCard class is the better final design because it owns the balance and prevents callers from bypassing the travel rule. The fare calculation should remain in a separate FarePolicy class or function to keep responsibilities focused.

TASK 8 – READ AND PRINT DATA SAFELY

8.1 Safe input using cin

The extraction operator >> attempts to read a value of the requested type. If the operator enters letters when an integer is expected, the stream enters a failure state.

Further extraction attempts will fail until the state is cleared. The program should test the result of every input operation rather than assuming that input is valid.

cin.fail() reports whether the last input operation failed. cin.clear() resets the stream state flags. The invalid characters must then be removed from the input buffer, commonly with cin.ignore(numeric_limits<streamsize>::max(), '\\n'). Clearing without discarding the bad input may cause the same characters to be read repeatedly.

```
#include <iostream>
#include <limits>

int readInteger(const char* prompt) {
   int value{};
   while (true) {
      std::cout << prompt;
      if (std::cin >> value) {
         return value;
      }
      std::cout << "Invalid input. Enter a whole number.\\n";
      std::cin.clear();
      std::cin.ignore(
         std::numeric_limits<std::streamsize>::max(), '\\n');
   }
}
```

8.2 Validating ranges

A successful numeric extraction does not prove that the value is meaningful. An hour of 28 is an integer but not a valid hour. A minute of 75 is also invalid. The program must validate hour between 0 and 23 and minute between 0 and 59. Zone input must match one of the supported zone codes. Recharge amounts must be positive. Validation should occur before fare calculation.

8.3 Formatting money with iomanip

The <iomanip> header provides manipulators for formatted output. setw(width) sets the minimum width for the next field. setfill(character) chooses the padding character. fixed and setprecision(2) display a floating-point value with two digits after the decimal point, but the recommended currency representation remains integer paise. For integer paise, print paise / 100 as rupees and paise % 100 as the two-digit fractional part.

```
#include <iomanip>
#include <iostream>

void printMoney(long long paise) {
   std::cout << "Rs " << paise / 100 << '.'
          << std::setw(2) << std::setfill('0')
          << paise % 100
```

```
            << std::setfill(' ');
    }
```

For example, 1005 paise prints as Rs 10.05. The setw manipulator applies only to the next output field, so it must be placed immediately before the paise remainder. setfill remains active until changed, so the example restores the fill character to a space after printing.

8.4 Neat journey report

Journey	Zone	Period	Fare	Status	Balance after trip
1	Inner	Off-peak	Rs 10.00	Approved	Rs 90.00
2	Outer	Peak	Rs 36.00	Approved	Rs 54.00
3	Middle	Off-peak	Rs 20.00	Denied	Rs 54.00

A report should show the journey number, zone, peak status, fare, approval or denial and resulting balance. A denied journey should show the unchanged balance. This makes the transaction understandable to both the passenger and the operator.

8.5 Alternative concept: getline and stoi

An alternative is to read an entire line with std::getline() and convert it using std::stoi() inside a try/catch block. This allows the program to reject trailing characters and provide line-based validation. Processing a line of length L takes O(L) time because the characters must be scanned, and the temporary string uses O(L) buffer space. Direct cin extraction also processes the characters in the input token and is O(L) for a token of length L, although it handles token boundaries differently.

getline is useful when input contains spaces or when the whole line should be validated as one record. cin >> is simpler for separate numeric fields. Both approaches can recover from invalid input, but getline plus stoi requires handling std::invalid_argument and std::out_of_range. Direct cin requires clearing the stream state and discarding the rest of the invalid line.

8.6 Evaluation

Use a reusable input-validation function, validate semantic ranges after parsing and print currency from integer paise. This avoids infinite input loops and ensures that the report reflects the actual card balance.

TASK 9 – IMPLEMENT AND TEST THE FARE LOGIC

9.1 Test strategy

Testing verifies that the implementation matches the stated rules. A useful test plan includes normal cases, boundary cases, invalid inputs and a small multi-journey sequence. Expected results should be calculated independently by hand before running the program. The actual result is then compared with the expected result and any difference is investigated.

9.2 Test cases

Test ID	Input / condition	Expected result	Reason
T01	Inner, off-peak	Rs 10.00	Base Inner fare
T02	Middle, off-peak	Rs 20.00	Base Middle fare
T03	Outer, off-peak	Rs 30.00	Base Outer fare
T04	Inner, 08:00	Rs 12.00	Peak start is included; 20% surcharge
T05	Inner, 10:00	Rs 10.00	Peak end is excluded
T06	Outer, 17:00	Rs 36.00	Second peak start is included
T07	Outer, 20:00	Rs 30.00	Second peak end is excluded
T08	Balance after fare exactly Rs 50	Approved	Rule allows remaining balance equal to Rs 50
T09	Balance after fare Rs 49.99	Denied	Remaining balance is below minimum
T10	Unknown zone	Rejected	Zone is outside valid enumeration
T11	Negative recharge	Rejected	Recharge must be positive
T12	Letters entered for hour	Input error and retry	Stream failure must be handled

For the boundary tests, the time should be represented with both hour and minute if the gate supports minute-level input. The hour-only version is sufficient to test exact whole-hour boundaries. The fare values in T04 and T06 are 1200 and 3600 paise respectively.

9.3 Balance boundary calculations

The minimum-balance condition is balance - fare >= 5000. If the balance is 6000 paise and the fare is 1000 paise, the remaining balance is exactly 5000 paise and the journey must be approved. If the balance is 5999 paise and the fare is 1000 paise, the remaining balance is 4999 paise and the journey must be denied. These two tests are important because they distinguish >= from >.

9.4 Five-journey hand calculation

Journey	Zone	Peak?	Fare (paise)	Fare (Rs)	Balance after journey
Start	—	—	—	—	10000 paise = Rs 100.00
1	Inner	No	1000	10.00	9000 = Rs 90.00
2	Outer	Yes	3600	36.00	5400 = Rs 54.00
3	Middle	No	2000	20.00	Denied; remains 5400
4	Inner	Yes	1200	12.00	Denied; remains 5400
5	Inner	No	1000	10.00	Denied; remains 5400

The first journey is approved and leaves Rs 90. The second journey is approved and leaves Rs 54. Each remaining fare would reduce the balance below Rs 50, so journeys 3, 4 and 5 are denied. The total collected is Rs 46.00. The card balance remains Rs 54.00 after the denied transactions.

9.5 Test harness approach

A table-driven test harness stores inputs and expected results in a collection and loops over them. This avoids writing a separate assertion for every test. If there are t test cases, the loop performs O(t) total work, while each individual fare calculation is O(1). The test table requires O(t) storage. Adding a boundary case becomes easy: add one row with its input and expected fare.

```
struct FareTest {
   Zone zone;
   bool peak;
   long long expected;
};

FareTest tests[] = {
   {Zone::Inner, false, 1000},
   {Zone::Middle, false, 2000},
   {Zone::Outer, false, 3000},
   {Zone::Inner, true, 1200},
   {Zone::Outer, true, 3600}
};

for (const FareTest& test : tests) {
   long long actual = fare(test.zone, test.peak);
   std::cout << (actual == test.expected ? "PASS" : "FAIL")
          << " expected=" << test.expected
          << " actual=" << actual << '\\n';
}
```

9.6 Evidence and limitations

The expected values above are derived from the assignment rules. The actual-results column must be filled after compiling and running the student's program. Screenshots should show the compiler command, test execution and result. Invalid text input should be tested interactively. The report must not describe a test as executed unless it was actually run.

9.7 Evaluation

The most important tests are the exact time boundaries and the exact Rs 50 remaining-balance boundary. A table-driven test suite provides repeatability and makes future fare changes easier to validate.

TASK 10 – COMPARE EXPECTATIONS WITH EXECUTION

10.1 Daily journey scale and collection totals

The assignment identifies three growth levels: one station with 20,000 journeys per day, one line with 500,000 journeys per day and the whole network with 3,000,000 journeys per day. The total collection depends on the mix of zones, peak journeys and concessions. To make a controlled comparison, the following table assumes every journey pays the same stated base fare and does not include concessions or surcharge.

Daily journeys	All at Rs 10	All at Rs 20	All at Rs 30
20,000	Rs 200,000	Rs 400,000	Rs 600,000
500,000	Rs 5,000,000	Rs 10,000,000	Rs 15,000,000
3,000,000	Rs 30,000,000	Rs 60,000,000	Rs 90,000,000

For the whole network, the maximum of these three uniform-fare examples is Rs 90 million, or 9 billion paise. A common signed 32-bit int cannot represent 9 billion, so accumulating paise in int may overflow. A signed long long can represent this total safely. The result for a real mixed network must be calculated from actual journey data.

10.2 Integer-paise versus double-rupee totals

If every fare is a whole number of rupees, converting each fare to paise and adding integer values gives an exact result within the integer range. A double total may also display the same result for many ordinary examples, but it does not provide the same exactness guarantee for arbitrary fractional fares. The comparison should be performed using the same sequence of input fares and the same rounding rule.

A reproducible experiment is to add 0.1 to a double repeatedly and compare it with an integer paise accumulator that adds 10 each time. Print the double with sufficient precision, for example std::setprecision(17), so that any binary representation difference can be observed. The report should include the exact measured output from the local compiler. The size of the error depends on the values and order of addition; it should not be guessed.

10.3 Compiler warnings with -Wall -Wextra

The command g++ -std=c++17 -Wall -Wextra program.cpp -o program enables a useful group of warnings. Possible warnings include signed/unsigned comparison, unused variables, missing return paths in non-void functions, suspicious conversions and unused parameters. The exact warnings depend on the source code and compiler version. A warning is not necessarily a compilation failure, but it should be understood and resolved where possible.

A signed/unsigned comparison warning means that operands of different signedness are being compared. An unused-variable warning means a declared value is not used. A control-reaches-end warning for a non-void function means some path may not return a value. The report should copy the actual diagnostic text from the student's build rather than inventing warnings.

10.4 Alternative accumulator types

Type	Typical size	Behaviour for 9 billion paise	Time per addition
int	Commonly 4 bytes	May overflow if signed 32-bit	O(1)
long long	Commonly 8 bytes	Safe for the stated total	O(1)
double	Commonly 8 bytes	Large enough in magnitude, but approximate representation	O(1)

The time per addition is O(1) for all three types. The main difference is numeric range and precision. int may overflow, long long provides exact integer paise within its range, and double has a wide magnitude range but cannot exactly represent every decimal fraction. The space per variable is constant, O(1).

10.5 Evidence-based comparison

For each scale, record the number of journeys, the fare distribution, the integer-paise total, the double total and the difference. Use the same input data for both calculations. If the data is generated, document the generation rule. If it is based on a real sample,

remove personal passenger information. The measured table should be completed after execution.

Journeys	Integer total (paise)	Double total (Rs)	Difference	Evidence status
20,000	Fill from run	Fill from run	Fill from run	To be measured
500,000	Fill from run	Fill from run	Fill from run	To be measured
3,000,000	Fill from run	Fill from run	Fill from run	To be measured

10.6 Evaluation

The evidence supports long long for money totals because it avoids the 32-bit overflow risk and retains exact integer-paise arithmetic. The report should separate mathematical calculations shown here from local compiler measurements that must be collected by running the experiment.

TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN

11.1 Recommended architecture

The final design should use a small set of classes with clear responsibilities. SmartCard owns the private balance and validates recharge and deduction. Journey stores the zone and time details. FarePolicy owns base fares, peak rules and concession policies. MetroGate or FareService coordinates the transaction and produces a result. This separation allows the fare policy to change without exposing the internal card state.

Component	Recommended representation	Reason
Zone	enum class	Named, strongly typed zone values
Money	long long paise	Exact currency arithmetic and large range
Fare constants	constexpr values or std::array	Single source of truth and compile-time checking
Time	validated hour/minute or minutes since midnight	Correct boundary comparisons
Card balance	private long long member	Protects minimum-balance invariant
Passenger type	enum class or polymorphic Passenger hierarchy	Supports concession policy
Journey processing	range-based for loop over journeys	Clear iteration over a collection

11.2 How a fare revision is performed

A fare revision should be made in the central fare configuration rather than by searching for numeric literals throughout the program. If the Inner fare changes, update its named fare entry. If the peak surcharge changes, update the surcharge constant. The

test suite must then be rerun, especially the peak and balance-boundary tests. The GitHub commit should describe the policy change and include the updated tests.

If the fare policy becomes configurable, it can be read from a validated configuration file at program startup. The file must be checked for missing zones, duplicate entries, negative fares, invalid percentages and malformed values. The program should not begin accepting passengers if the fare configuration is incomplete or invalid.

11.3 Limitations of the design

- The original assignment does not state the exact student or senior-citizen concession percentage.
- The order in which concession and peak surcharge are applied is not specified.
- The original system describes three zones but does not fully specify station-to-zone mapping or how the farthest zone is determined.
- The example implementation uses a simplified in-memory card balance and does not include a database, authentication or transaction rollback.
- A real metro system must handle simultaneous gate requests, network failures, duplicate transactions, audit logs and secure card authentication.
- Integer arithmetic still needs a documented rounding rule for percentages that produce fractional paise.
- Overflow checks are required if balances or aggregates can grow beyond the selected type's range.

11.4 Suitability for the whole network

The fare calculation itself is constant-time, O(1), because it performs a fixed number of lookups, comparisons and arithmetic operations. Processing J journeys takes O(J) time. A collection containing J journeys uses O(J) storage if all journeys are stored before processing; streaming journeys one at a time can reduce additional processing memory to O(1), apart from the card and transaction state.

The whole-network scale of three million journeys per day does not change the asymptotic complexity. It does make integer range, performance testing, data integrity and operational reliability more important. The design should be benchmarked with realistic data and tested under concurrent access before deployment.

11.5 Alternative concept: configuration file

Reading z zone fares from a configuration file requires O(z) time at startup and O(z) memory to store the values. Each journey can still use O(1) fare lookup when the table is held in an array or map with appropriate access. The benefit is that an authorised

fare revision can be deployed by updating configuration rather than recompiling the program. The risks are malformed configuration, accidental edits and inconsistent versions across gates.

Compiled constexpr values require a rebuild and redeployment for every revision, but they are checked at compile time and cannot be changed accidentally at runtime. A configuration file is more flexible but must be validated, versioned, secured and rolled out consistently. For a small educational program, constexpr is the simpler choice. For a production network with frequent policy changes, a signed and validated configuration service may be more appropriate.

11.6 Final recommendation

Retain the object-oriented structure, use integer paise and long long, centralise fare values, and preserve a strict SmartCard balance invariant. Use explicit time intervals and a table-driven regression suite. The design is suitable as a foundation for the full network, but production deployment requires persistence, concurrency control, secure card identification and operational monitoring.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN

12.1 Changed requirements

The metro introduces three new requirements: a fourth Airport zone with a base fare of Rs 60, a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones. These changes affect the fare table, zone representation, fare calculation order and passenger entitlement model.

A. Impact analysis

The Zone enum must gain Airport. The base fare table must gain 6000 paise. Any switch statement that selects a fare must gain an Airport case. The number of fare entries and validation rules must be updated. Tests must include Airport peak, Airport off-peak, weekend and monthly-pass journeys.

The fare calculation must distinguish weekday peak, weekday off-peak and weekend. The new rule says the weekend discount does not combine with the peak surcharge. A clear policy is therefore: on weekends, apply a 10% discount and do not add the 20% peak surcharge; on weekdays, apply the 20% surcharge only during peak periods. The assignment does not state whether the weekend discount applies to every passenger

category or how it interacts with a concession, so that interaction must be confirmed before production.

The monthly pass introduces a separate entitlement. A pass must record its validity period and the two selected zones. If a journey's zone is covered by an active pass, the base fare may be zero for that journey. The prompt says unlimited journeys in two chosen zones; it does not specify whether a journey that passes through a chosen zone but ends in another zone is covered. The example design treats the journey's farthest zone as the zone checked against the pass.

B. Evaluate the original switch and SmartCard designs

The original switch-based design can handle Airport by adding another case. For four fixed zones, this is still manageable. However, every zone addition requires edits to the enum, switch and tests. A fare table indexed by a validated zone index becomes more convenient when the number of zones grows. The SmartCard class can still protect the stored balance, but a monthly pass is a different entitlement and should not be forced into the card's balance field.

A separate MonthlyPass class is preferable. It stores the valid-from and valid-until dates and the two covered zones. It exposes covers(zone, date), which returns true only when the pass is active and the zone is covered. FarePolicy checks the pass before calculating a cash fare. This keeps stored-value balance and pass entitlement as separate concepts.

C. Revised design and implementation

The following implementation demonstrates the updated fare rule using integer paise. It assumes weekend status is supplied by the caller, that weekend discount replaces the peak surcharge, and that the monthly pass covers the farthest zone of the journey. Date validity is represented by an active flag for this compact demonstration; a production implementation should compare actual dates.

```
#include <array>
#include <iostream>

enum class Zone { Inner, Middle, Outer, Airport };

constexpr std::array<long long, 4> BASE_FARE_PAISE = {
   1000, 2000, 3000, 6000
};
constexpr int PEAK_RATE = 20;
constexpr int WEEKEND_DISCOUNT = 10;
constexpr long long MIN_BALANCE = 5000;

int zoneIndex(Zone zone) {
```

```
    switch (zone) {
        case Zone::Inner: return 0;
        case Zone::Middle: return 1;
        case Zone::Outer: return 2;
        case Zone::Airport: return 3;
    }
    return -1;
}

class MonthlyPass {
private:
    bool active;
    Zone firstZone;
    Zone secondZone;

public:
    MonthlyPass(bool isActive, Zone a, Zone b)
        : active(isActive), firstZone(a), secondZone(b) {}

    bool covers(Zone zone) const {
        return active && (zone == firstZone || zone == secondZone);
    }
};

long long calculateRevisedFare(
    Zone zone, bool peak, bool weekend, const MonthlyPass& pass) {

    if (pass.covers(zone)) {
        return 0;
    }

    int index = zoneIndex(zone);
    if (index < 0) {
        return -1;
    }

    long long fare = BASE_FARE_PAISE[index];

    // Weekend discount replaces, and does not combine with, peak surcharge.
    if (weekend) {
        fare -= fare * WEEKEND_DISCOUNT / 100;
    } else if (peak) {
        fare += fare * PEAK_RATE / 100;
    }

    return fare;
}

class SmartCard {
private:
    long long balance;

public:
    explicit SmartCard(long long initial) : balance(initial >= 0 ? initial : 0) {}
```

```
    bool pay(long long fare) {
        if (fare < 0 || balance < fare ||
            balance - fare < MIN_BALANCE) {
            return false;
        }
        balance -= fare;
        return true;
    }

    long long getBalance() const { return balance; }
};

int main() {
    MonthlyPass pass(true, Zone::Inner, Zone::Middle);
    SmartCard card(20000);

    struct TestTrip {
        Zone zone;
        bool peak;
        bool weekend;
    };

    TestTrip trips[] = {
        {Zone::Inner, true, false},     // covered by pass: Rs 0
        {Zone::Outer, true, false},     // Rs 36
        {Zone::Airport, false, true}    // Rs 54 after 10% discount
    };

    for (const auto& trip : trips) {
        long long fare = calculateRevisedFare(
            trip.zone, trip.peak, trip.weekend, pass);

        if (fare >= 0 && card.pay(fare)) {
            std::cout << "Approved, fare paise = " << fare
                      << ", balance paise = " << card.getBalance()
                      << '\\n';
        } else {
            std::cout << "Denied, balance paise = "
                      << card.getBalance() << '\\n';
        }
    }
}
```

12.4 Worked examples

Case	Rule applied	Calculation	Result
Inner weekday peak with pass covering Inner	Pass entitlement	Covered zone gives zero fare	Rs 0.00
Outer weekday peak	20% peak surcharge	3000 + (3000 × 20 / 100) = 3600 paise	Rs 36.00
Airport weekend	10% discount; no peak surcharge	6000 − (6000 × 10 / 100) = 5400 paise	Rs 54.00
Airport weekday off-peak	Base fare only	6000 paise	Rs 60.00
Airport weekday peak	20% surcharge	6000 + 1200 = 7200 paise	Rs 72.00

These examples make the precedence of policy rules explicit. A weekend trip receives the weekend discount and does not receive the peak surcharge even if the clock time is within a weekday peak interval. A monthly pass covering the selected zone overrides the cash fare in this simplified model. The actual business owner must confirm whether pass coverage has priority over weekend discounts and concessions.

12.5 Revised test cases

Test	Input	Expected fare / result
D01	Airport, weekday off-peak, no pass	Rs 60.00
D02	Airport, weekday peak, no pass	Rs 72.00
D03	Airport, weekend, no pass	Rs 54.00
D04	Airport, weekend peak time	Rs 54.00; no peak surcharge
D05	Inner zone covered by active pass	Rs 0.00
D06	Outer zone not covered by pass	Normal fare policy applies
D07	Pass inactive	Normal fare policy applies
D08	Invalid zone value	Rejected

12.6 D. Evidence-based defence

The recommended decision is MODIFY rather than completely redesign the original system. Retain SmartCard because it still represents stored value and protects the minimum-balance rule. Retain integer paise because the new fares and discounts can be calculated exactly for the supplied values. Modify Zone and the fare table to include Airport. Introduce a separate FarePolicy branch for weekend discount and peak surcharge. Add MonthlyPass as a separate class because it represents a time-limited entitlement rather than stored money.

The revised fare table scales more conveniently than a long switch when zones are added. The zoneIndex function validates the enum and maps it to a fixed array index. The monthly pass is separated from SmartCard so that the balance logic remains simple. The test suite must verify that weekend and peak rules do not combine and that only the two selected zones are free while the pass is active.

The design should be defended with actual execution evidence: compile with warnings enabled, run each revised test, record expected and actual values, and capture output screenshots. The worked values in this report are arithmetic expectations; they are not a claim that the code has been executed on the student's computer.

12.7 Trade-offs and limitations

The modified design introduces additional classes and policy branches, which increases code size and test requirements. In return, it separates stored-value payment, fare calculation and pass entitlement. A fully configurable fare engine may be more appropriate if zones and policy combinations grow significantly. Before production, the metro must define concession ordering, pass date validity, refund rules, cross-zone pass coverage and the treatment of journeys that begin in one zone and end in another.

12.8 Final decision

Modify the original design. The SmartCard class and integer-paise representation remain valid. The fare table and zone enumeration must be extended. Weekend discounts and monthly passes should be modelled as separate policy and entitlement components. This is a controlled extension of the original architecture and avoids rewriting stable parts of the system.

OVERALL CONCLUSION

The Namma Metro Smart-Card Fare System demonstrates why correctness, data representation and software structure are important in a transaction-processing application. The original fare rules can be implemented with a small number of classes and constant-time calculations, but the implementation must handle boundary times, minimum balances, invalid input and large network totals carefully.

The report recommends integer paise stored in long long, constexpr fare constants, enum class for zones, explicit peak-hour intervals, validated input and a SmartCard class with private balance. These choices reduce rounding risk, prevent accidental invalid states and make fare revisions easier. The procedural approach is useful for understanding the logic, while object-oriented encapsulation is preferable for protecting card state.

Testing must include normal fares, peak fares, exact time boundaries, the exact Rs 50 remaining-balance boundary, invalid zones, invalid recharge amounts and invalid textual input. Complexity analysis shows that one fare calculation is O(1) and processing J journeys is O(J). The system's growth to three million journeys per day does not alter the asymptotic complexity, but it makes numeric range, reliability and performance measurement essential.

For the Day-4 design challenge, the recommended response is to modify rather than discard the original architecture. Add the Airport zone to the fare table, apply the weekend discount instead of the peak surcharge on weekends, and introduce a separate

MonthlyPass class. The final design remains modular and can be extended further when the metro provides precise concession and pass-coverage rules.

SUBMISSION CHECKLIST

- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1.
- Create one folder for each task and include the C++ source files, input files and output screenshots.
- Include a README.md with name, register number, compiler version, compile command, run instructions and result notes.
- Compile with g++ -std=c++17 -Wall -Wextra and resolve or explain warnings.
- Commit progressively, with at least one commit for each task.
- Add the working GitHub repository link to the cover page and the assignment submission.
- Run every Complete the Code program and capture its actual output.
- Write output predictions before running the Predict the Output programs.
- Replace all 'to be measured' evidence fields with genuine measurements from the local system.

APPENDIX – CONSOLIDATED C++ REFERENCE IMPLEMENTATION

The following compact reference implementation combines the central original three-zone fare rules. It is intended as a clean starting point for execution and further extension. It does not invent a student or senior concession percentage. The program reads a numeric zone code and time, validates them, calculates the fare in paise and protects the card's minimum balance.

```
#include <iomanip>
#include <iostream>
#include <limits>

enum class Zone { Inner = 1, Middle = 2, Outer = 3 };

constexpr long long MIN_BALANCE_PAISE = 5000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;

bool isPeakHour(int hour) {
   return (hour >= 8 && hour < 10) ||
          (hour >= 17 && hour < 20);
}

long long baseFare(Zone zone) {
   switch (zone) {
      case Zone::Inner:  return 1000;
      case Zone::Middle: return 2000;
      case Zone::Outer:  return 3000;
```

```
    }
    return -1;
}

long long calculateFare(Zone zone, bool peak) {
    long long fare = baseFare(zone);
    if (fare < 0) return -1;
    if (peak) {
        fare += fare * PEAK_SURCHARGE_PERCENT / 100;
    }
    return fare;
}

class SmartCard {
private:
    long long balancePaise;

public:
    explicit SmartCard(long long initialBalance)
        : balancePaise(initialBalance >= 0 ? initialBalance : 0) {}

    bool canTravel(long long farePaise) const {
        return farePaise >= 0 &&
               balancePaise >= farePaise &&
               balancePaise - farePaise >= MIN_BALANCE_PAISE;
    }

    bool deduct(long long farePaise) {
        if (!canTravel(farePaise)) return false;
        balancePaise -= farePaise;
        return true;
    }

    long long balance() const { return balancePaise; }
};

void printMoney(long long paise) {
    std::cout << "Rs " << paise / 100 << '.'
              << std::setw(2) << std::setfill('0')
              << paise % 100 << std::setfill(' ');
}

int main() {
    int zoneInput{};
    int hour{};
    std::cout << "Enter zone (1=Inner, 2=Middle, 3=Outer): ";
    if (!(std::cin >> zoneInput)) {
        std::cerr << "Invalid zone input.\\n";
        return 1;
    }

    std::cout << "Enter hour (0-23): ";
    if (!(std::cin >> hour)) {
        std::cerr << "Invalid hour input.\\n";
        return 1;
```

```
    }

    if (hour < 0 || hour > 23) {
        std::cerr << "Hour must be between 0 and 23.\\n";
        return 1;
    }

    if (zoneInput < 1 || zoneInput > 3) {
        std::cerr << "Unknown zone.\\n";
        return 1;
    }

    Zone zone = static_cast<Zone>(zoneInput);
    long long fare = calculateFare(zone, isPeakHour(hour));

    SmartCard card(10000);
    if (card.deduct(fare)) {
        std::cout << "Approved. Fare: ";
        printMoney(fare);
        std::cout << ", Balance: ";
        printMoney(card.balance());
        std::cout << '\\n';
    } else {
        std::cout << "Denied (low balance). Balance: ";
        printMoney(card.balance());
        std::cout << '\\n';
    }
    return 0;
}
```

Compile command: g++ -std=c++17 -Wall -Wextra -pedantic metro_fare.cpp -o metro_fare

Run command on Windows Command Prompt: metro_fare.exe

Run command on Linux or macOS terminal: ./metro_fare

Before submission, replace the blank cover-page details, add the real GitHub URL, execute each task program independently, insert screenshots where required by the faculty and fill in all local measurement fields. The assignment brief requires individual execution and progressive GitHub commits; this report is a written answer document and does not substitute for those practical submission requirements.